

케라스 신경망 모델 구 성법

주요 내용

- 다양한 신경망 모델 구성법
- 신경망 모델과 층의 재활용
- 신경망 모델 훈련 옵션: 콜백과 텍서보드

신경망 모델 구성법 1: Sequential 모델 활용

- Sequential 모델은 층으로 스택을 쌓아 만든 모델이며 가장 단순함
- 한 종류의 입력값과 한 종류의 출력값만 사용 가능
- 순전파: 지정된 층의 순서대로 적용

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])
```

summary() 메서드

```
>>> model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Input() 함수

```
model = keras.Sequential([
    keras.Input(shape=(784,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])
```

```
>>> model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 64)	50,240
dense_5 (Dense)	(None, 10)	650

Total params: 50,890 (198.79 KB)

Trainable params: 50,890 (198.79 KB)

Non-trainable params: 0 (0.00 B)

신경망 모델 구성법 2: 함수형 API

```
inputs = keras.Input(shape=(3,), name="my_input")           # 입력층
features = layers.Dense(64, activation="relu")(inputs)       # 은닉층
outputs = layers.Dense(10, activation="softmax")(features)   # 출력층
model = keras.Model(inputs=inputs, outputs=outputs)          # 모델 지정
```

```
>>> model.summary()
```

Model: "functional_6"

Layer (type)	Output Shape	Param #
my_input (InputLayer)	(None, 784)	0
dense_10 (Dense)	(None, 64)	50,240
dense_11 (Dense)	(None, 10)	650

Total params: 50,890 (198.79 KB)

Trainable params: 50,890 (198.79 KB)

Non-trainable params: 0 (0.00 B)

다중 입력, 다중 출력 모델

다중 입력과 다중 출력을 지원하는 모델을 구성하는 방법을 예제를 이용하여 설명한다.

- 입력층: 세 개
- 은닉층: 두 개
- 출력층: 두 개


```
vocabulary_size = 10000    # 사용빈도 1만등 이내 단어 사용
num_tags = 100             # 태그 수
num_departments = 4        # 부서 수

# 입력층: 세 개
title = keras.Input(shape=(vocabulary_size,), name="title")
text_body = keras.Input(shape=(vocabulary_size,), name="text_body")
tags = keras.Input(shape=(num_tags,), name="tags")

# 은닉층
features = layers.Concatenate()([title, text_body, tags]) # shape=(None,
10000+10000+100)
features = layers.Dense(64, activation="relu")(features)

# 출력층: 두 개
priority = layers.Dense(1, activation="sigmoid", name="priority")(features)
department = layers.Dense(
    num_departments, activation="softmax", name="department")(features)

# 모델 빌드: 입력값으로 구성된 입력값 리스트와 출력값으로 구성된 출력값 리스트
사용
model = keras.Model(inputs=[title, text_body, tags], outputs=[priority,
department])
```


모델 훈련

```
import numpy as np

# 샘플 수
num_samples = 1280

# 입력 텐서 3 개 무작위 생성
title_data = np.random.randint(0, 2, size=(num_samples, vocabulary_size))
text_body_data = np.random.randint(0, 2, size=(num_samples, vocabulary_size))
tags_data = np.random.randint(0, 2, size=(num_samples, num_tags))    # 멀티-핫-인코딩

# 타깃 텐서 2 개 무작위 생성
priority_data = np.random.random(size=(num_samples, 1))
department_data = np.random.randint(0, 2, size=(num_samples, num_departments))
# 멀티-핫-인코딩

model.fit([title_data, text_body_data, tags_data],
          [priority_data, department_data],
          epochs=10)
```

모델 평가

```
model.evaluate([title_data, text_body_data, tags_data],  
               [priority_data, department_data])
```

모델 활용

[illegible]

- 우선 순위 예측값: 0과 1사이의 확률값

```
>>> priority_preds  
array([[1.],  
       [1.],  
       [1.],  
       ...,  
       [1.],  
       [1.],  
       [1.]], dtype=float32)
```

- 처리 부서 예측값: 각 부서별 적정도를 가리키는 확률값

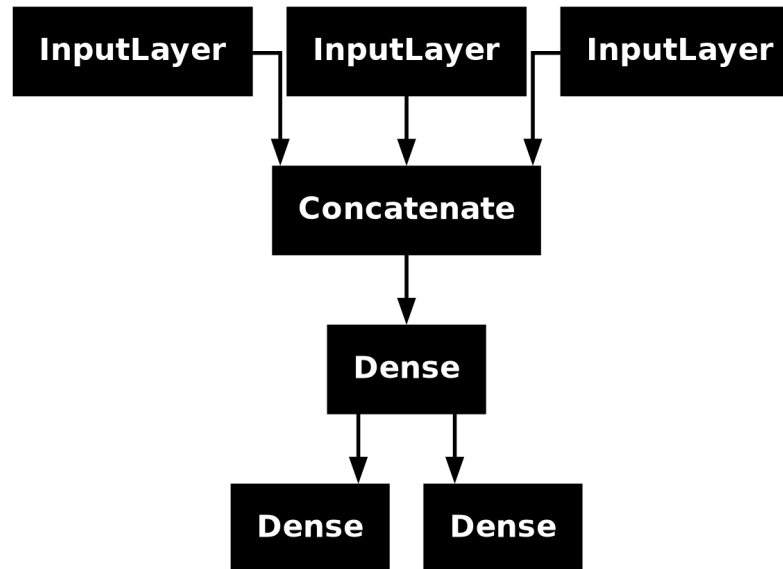
```
>>> department_preds
array([[1.0000000e+00, 5.0770291e-28, 4.8780390e-08, 1.4980437e-14],
       [1.0000000e+00, 8.9238867e-31, 4.3726455e-08, 1.0175944e-14],
       [1.0000000e+00, 1.6067065e-29, 5.4092455e-09, 2.2488301e-14],
       ...,
       [1.0000000e+00, 1.8711068e-29, 2.2684986e-08, 5.9480673e-14],
       [1.0000000e+00, 4.0235123e-29, 1.5713432e-08, 9.7684718e-14],
       [1.0000000e+00, 2.8694353e-30, 7.1231565e-10, 6.3928655e-15]],
      dtype=float32)
```

각각의 요구사항을 처리해야 하는 부서는 `argmax()` 메서드로 확인된다.

```
>>> department_preds.argmax()  
array([0, 0, 0, ..., 0, 0, 0])
```


신경망 모델 구조 그래프

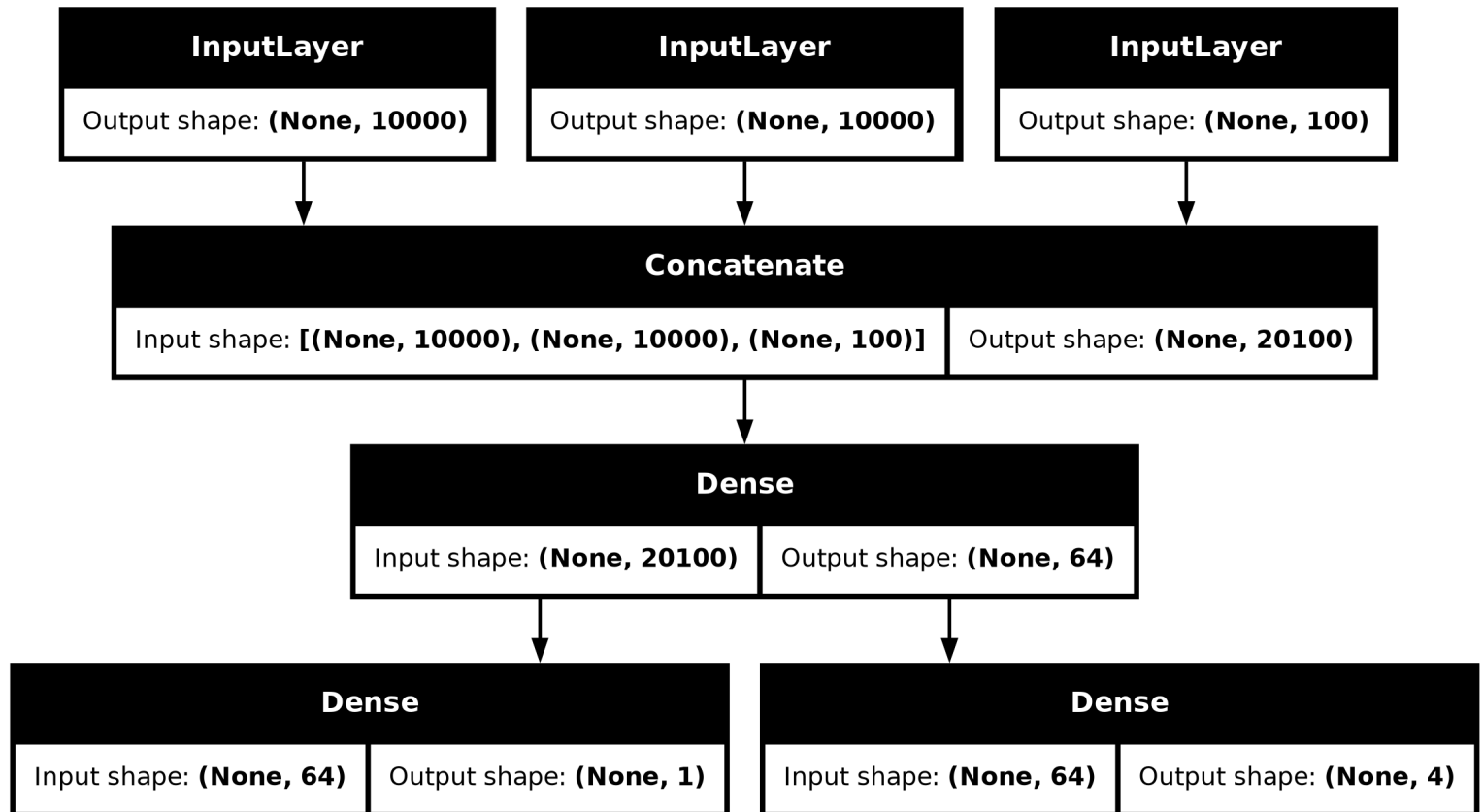
```
>>> keras.utils.plot_model(model, "ticket_classifier.png")
```



`plot_model()` 함수 사용 준비 사항

- `pydot` 모듈 설치: `pip install pydot`
- `graphviz` 프로그램 설치: <https://graphviz.gitlab.io/download/>
- 구글 코랩에서는 기본으로 지원됨.

```
>>> keras.utils.plot_model(model, "ticket_classifier_with_shape_info.png",  
show_shapes=True)
```



신경망 모델 구성법 3: 서브클래싱

- `keras.Model` 클래스를 상속하는 모델 클래스를 직접 선언
- `__init__()` 메서드(생성자): 은닉층과 출력층으로 사용될 층 객체 지정
- `call()` 메서드: 층을 연결하는 과정 지정. 즉, 입력값으로부터 출력값을 만들어내는 순전파 과정 묘사.

예제: 고객 요구사항 처리 모델

```
class CustomerTicketModel(keras.Model):
    def __init__(self, num_departments):
        super().__init__()
        self.concat_layer = layers.Concatenate()
        self.mixing_layer = layers.Dense(64, activation="relu")
        self.priority_scorer = layers.Dense(1, activation="sigmoid")
        self.department_classifier = layers.Dense(
            num_departments, activation="softmax")

    def call(self, inputs):
        title = inputs["title"]
        text_body = inputs["text_body"]
        tags = inputs["tags"]

        features = self.concat_layer([title, text_body, tags])
        features = self.mixing_layer(features)
        priority = self.priority_scorer(features)
        department = self.department_classifier(features)
        return priority, department

model = CustomerTicketModel(num_departments=4)
```

서브클래싱 기법의 장단점

- 장점

- `call()` 함수를 이용하여 층을 임의로 구성할 수 있다.
- `for` 반복문 등 파이썬 프로그래밍 모든 기법을 적용할 수 있다.

- 단점

- 모델 구성을 전적으로 책임져야 한다.
- 모델 구성 정보가 `call()` 함수 외부로 노출되지 않아서 앞서 보았던 그래프 표현을 사용할 수 없다.

혼합 신경망 모델 구성법

모델 vs. 층

- `keras.Model` 이 `keras.layers.Layer` 의 자식 클래스
- 이미 선언된 모델을 다른 모델에서 하나의 층으로 활용 가능
- 층 클래스와는 다르게 모델 클래스는 `fit()`, `evaluate()`, `predict()` 메서드를 함께 지원하여 모델의 훈련, 평가, 활용을 담당하도록 함.

예제: 서브클래싱 모델을 함수형 모델에 활용하기

```
class Classifier(keras.Model):  
  
    def __init__(self, num_classes=2):  
        super().__init__()  
        if num_classes == 2:  
            num_units = 1  
            activation = "sigmoid"  
        else:  
            num_units = num_classes  
            activation = "softmax"  
        self.dense = layers.Dense(num_units, activation=activation)  
  
    def call(self, inputs):  
        return self.dense(inputs)
```

```
inputs = keras.Input(shape=(3,)) # 입력층  
features = layers.Dense(64, activation="relu")(inputs) # 은닉층  
outputs = Classifier(num_classes=10)(features) # 출력층  
  
model = keras.Model(inputs=inputs, outputs=outputs)
```

예제: 함수형 모델을 서브클래싱 모델에 활용하기

```
inputs = keras.Input(shape=(64,))
outputs = layers.Dense(1, activation="sigmoid")(inputs)
binary_classifier = keras.Model(inputs=inputs, outputs=outputs)
```

```
class MyModel(keras.Model):

    def __init__(self, num_classes=2):
        super().__init__()
        self.dense = layers.Dense(64, activation="relu")
        self.classifier = binary_classifier

    def call(self, inputs):
        features = self.dense(inputs)
        return self.classifier(features)
```

신경망 모델의 구성, 훈련, 평가, 예측

- 딥러닝 신경망 모델의 훈련은 한 번 시작되면 훈련이 종료될 때까지 어떤 간섭도 받지 않는다.
- 다만, 훈련 진행과정을 관찰_{monitoring}할 수 있을 뿐이다.
- 훈련 과정 동안 관찰할 수 있는 내용은 일반적으로 다음과 같다.
 - 에포크별 손실값
 - 에포크별 평가지표

콜백

- 훈련 기록 작성
 - 훈련 에포크마다 보여지는 손실값, 평가지표 등 관리
 - `keras.callbacks.CSVLogger` 클래스 활용.
- 훈련중인 모델의 상태 저장
 - 훈련 중 가장 좋은 성능의 모델(의 상태) 저장
 - `keras.callbacks.ModelCheckpoint` 클래스 활용
- 훈련 조기 종료
 - 검증셋에 대한 손실이 더 이상 개선되지 않는 경우 훈련을 종료 시키기
 - `keras.callbacks.EarlyStopping` 클래스 활용
- 하이퍼 파라미터 조정
 - 학습률 동적 변경 지원
 - `keras.callbacks.LearningRateScheduler` 또는 `keras.callbacks.ReduceLROnPlateau` 클래스 활용

예제

```
callbacks_list = [  
    keras.callbacks.EarlyStopping(  
        monitor="val_accuracy",  
        patience=2,  
    ),  
    keras.callbacks.ModelCheckpoint(  
        filepath="checkpoint_path",  
        monitor="val_loss",  
        save_best_only=True,  
    )  
]
```

에포크를 50 정도로 크게 잡고 훈련을 실행하면 조기 종료 기능에 의해 10번 에포크 정도에서 훈련이 종료된다.

```
def get_mnist_model():
    inputs = keras.Input(shape=(28 * 28,))
    features = layers.Dense(512, activation="relu")(inputs)
    features = layers.Dropout(0.5)(features)
    outputs = layers.Dense(10, activation="softmax")(features)
    model = keras.Model(inputs, outputs)
    return model

model = get_mnist_model()

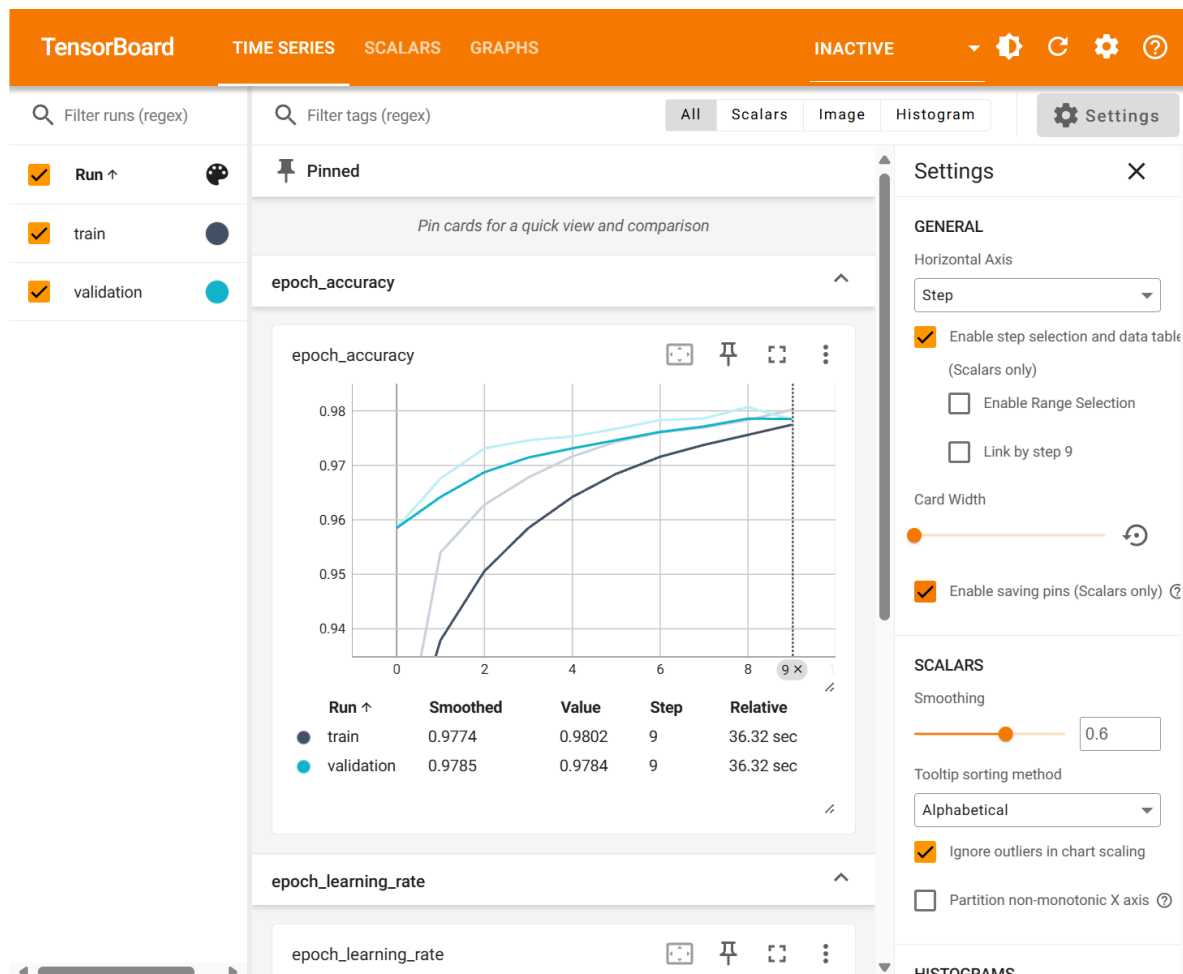
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

model.fit(train_images, train_labels,
          epochs=50,
          callbacks=callbacks_list,
          validation_data=(val_images, val_labels))
```

```
Epoch 1/50
1563/1563 ----- 4s 2ms/step - accuracy:
0.8635 - loss: 0.4526 - val_accuracy: 0.9594 - val_loss: 0.1473
Epoch 2/50
1563/1563 ----- 2s 2ms/step - accuracy:
0.9526 - loss: 0.1668 - val_accuracy: 0.9671 - val_loss: 0.1173
Epoch 3/50
1563/1563 ----- 3s 2ms/step - accuracy:
0.9616 - loss: 0.1319 - val_accuracy: 0.9718 - val_loss: 0.1117
...
...
Epoch 9/50
1563/1563 ----- 3s 2ms/step - accuracy:
0.9792 - loss: 0.0714 - val_accuracy: 0.9786 - val_loss: 0.0929
Epoch 10/50
1563/1563 ----- 2s 1ms/step - accuracy:
0.9804 - loss: 0.0733 - val_accuracy: 0.9782 - val_loss: 0.0957
Epoch 11/50
1563/1563 ----- 2s 2ms/step - accuracy:
0.9816 - loss: 0.0685 - val_accuracy: 0.9782 - val_loss: 0.0924
<keras.src.callbacks.history.History at 0x7f3251f342f0>
```


텐서보드

- 신경망 모델 구조 시각화
- 손실값, 정확도 등의 변화 시각화
- 가중치, 편향 텐서 등의 변화 히스토그램
- 이미지, 텍스트, 오디오 데이터 시각화
- 기타 다양한 기능 제공



텐서보드는 `TensorBoard` 콜백 클래스를 활용한다.

- `log_dir`: 텐서보드 서버 실행에 필요한 데이터 저장소 지정

```
tensorboard = keras.callbacks.TensorBoard(  
    log_dir="./tensorboard_log_dir",  
)  
  
model.fit(train_images, train_labels,  
          epochs=10,  
          validation_data=(val_images, val_labels),  
          callbacks=[tensorboard])
```

- 주피터 노트북에서

```
%load_ext tensorboard  
%tensorboard --logdir ./tensorboard_log_dir
```

- 터미널에서

```
$ tensorboard --logdir ./tensorboard_log_dir
```